

嵌入式電腦視覺系統

用 ML / DL 完成猜拳辨識 (RPS) : 含研究方法、評估指標、Android + Colab 雙平台部署

謝昇憲 George Hsieh

學習路線大綱

第 1 章：本講課程定位



第 2 章：研究 vs 功能實作 (研究生最該分清的事)



第 3 章：評估指標完整講解



第 4 章：資料集與前處理 (Colab)



第 5 章：路線 B | HoG + Linear SVM (訓練 + Android Chaquopy 部署)



第 6 章：路線 C | Tiny CNN (訓練 + Colab Local Runtime 部署)



第 7 章：路線 D | MobileNetV2 Transfer Learning (訓練 + Colab Local Runtime 部署)



第 8 章：兩種部署平台的對比與工程意義



第 9 章：三方法 × 傳統 CV 在系統上實測對比 (研究級對比)



第 10 章：本講作業與常見錯誤排除

第 1 章 本講課程定位

1.1 從「規則法」進到「會學的方法」

上一講「傳統 CV 規則法做 RPS」我們親手用了 W6 形態學 + W7 凸缺陷數，預期正確率落在 60% - 80%。

那個天花板來自一個本質限制：規則法的「特徵」是人寫死的，遇到沒考慮到的情境就無法泛化。

本講的核心問題是：

當「特徵」由演算法從資料學出來，會比人手刻的好多少？代價是什麼？

1.2 本講三條路

路線 B | 傳統 ML : HoG + Linear SVM — 手工特徵 + 學習式決策邊界

路線 C | DL 自訓 : Tiny CNN — 端到端從像素學特徵，但模型小、可手調

路線 D | DL 遷移學習 : MobileNetV2 — 用 ImageNet 預訓練權重，微調分類層

三條路會跑在同一個資料集 (Sign Language Digits, 只保留 0/2/5) 上，最後在「正確率、推論延遲、模型大小」三個維度對比。

1.3 部署平台分工（這是本講的關鍵工程決定）

「能訓練」≠「能部署」。三條路的部署平台必須依「Chaquopy 是否有對應 Python wheel」決定：

路線 B (HoG + SVM) → Android Chaquopy
scikit-learn / scikit-image / joblib 都有 cp310 Android wheel
→ 可以直接塞進手機跑

路線 C / D (Tiny CNN / MobileNetV2 → TFLite) → Colab + Local Runtime + 本地 Webcam
tflite-runtime 在 Chaquopy 官方 repo 只有 cp38 wheel
Chaquopy 17 (Python 3.10) 安裝會失敗
→ 改用 Colab 連本地 Jupyter , webcam 推論

下一講 (YOLO + Tracking) → Android Java TFLite API + AAR
完全避開 Python wheel 限制

1.4 為什麼要這樣分工？

原因有三：

避免「之前 YOLO 那種掛掉」的部署失敗：Chaquopy 沒有的 wheel 就不要硬塞
讓學生看到「不同模型用不同部署路徑」是工程常態，不是失敗
Colab Local Runtime 你已經教過串流架構，本講直接用

1.5 學習目標

完成本講，研究生應該能做到：

理解「功能實作」與「研究級實作」的差別，並能寫出研究級報告
看得懂 Confusion Matrix、Precision、Recall、F1、推論延遲
用 sklearn 訓練 HoG + Linear SVM 並部署到 Android Chaquopy
用 Keras 訓練 Tiny CNN 與 MobileNetV2，並輸出 TFLite
在 Colab Local Runtime + Webcam 上做即時 demo
在「正確率、延遲、模型大小」三維度做定量對比，產出研究級對照表

第 2 章 研究 vs 功能實作 | 研究生最該分清的事

2.1 「能 work」≠「能交研究報告」

多數同學交作業時的心態是：「我有跑出來，所以我做完了」。

這個心態對「工程功能交付」是足夠的，但對「研究」遠遠不夠。

研究級報告必須回答「在什麼資料上、性能多好、比誰好、系統 spec 是什麼」這四個問題。
「我有跑」不算答案。

2.2 功能實作的三個盲點

2.2.1 盲點一：「我跑了 1 張圖，準確！」

這句話沒有「test set」、沒有「樣本數」、沒有「定量指標」。

研究級的對應陳述：

「在 $N=200$ 張獨立測試集上， $\text{accuracy} = 92.5\%$ (95% CI: 89.1% – 95.2%)， $\text{precision/recall/F1} = \dots$ 」

2.2.2 盲點二：「我在我家光線下準確」

這句話沒有 spec、沒有 generalization 討論。

研究級的對應陳述：

「實測 spec：iPhone 13、室內螢光燈、約 500 lux、單手、距離鏡頭 30-50 cm、白色背景。脫離此 spec 後性能下降 X%。」

2.2.3 盲點三：「我比之前快」

沒有 baseline、沒有比較、沒有控制變因。

研究級的對應陳述：

「在相同 PC、相同 Webcam、相同光線下，我的方法平均延遲 X ms ($n=100$)，baseline A 為 Y ms，差異 P 值 < 0.01 。」

2.3 研究級系統必須回答的四個問題

2.3.1 問題一 | 在什麼資料上？

資料集名稱、來源、大小（樣本數）

資料分割策略（train / val / test 各幾筆，是否 stratify）

資料是否獨立同分佈（IID）？有沒有 data leakage？

資料增強策略（rotation、flip、scale 等）的記錄

2.3.2 問題二 | 性能多好？

Accuracy、Precision、Recall、F1（多個指標，不只一個）

Confusion Matrix（看到底哪兩類最常搞混）

置信區間 95% CI（樣本數有限時必須給）

在不同子集上的表現（譬如不同手勢、不同光線）

2.3.3 問題三 | 比誰好/壞？

至少一個 baseline（譬如本講的「傳統 CV 規則法」就是 ML/DL 的 baseline）

SOTA（state-of-the-art）對比（譬如跟近期 paper 比）

Ablation study：拿掉某一個 component，性能掉多少？

統計顯著性測試（譬如 paired t-test、McNemar test）

2.3.4 問題四 | 系統 spec 是什麼？

硬體：CPU 型號、GPU 型號、RAM、儲存空間

軟體：作業系統、Python 版本、套件版本

運行條件：解析度、相機型號、光線、距離

效能 spec：平均延遲、最大延遲、FPS、記憶體峰值、模型大小

2.4 從 demo 升級到 paper 的標準清單

交研究級報告前，請逐項勾選：

- ✓ 報告有「實驗設定」章節，列明所有 spec
- ✓ 有 test set (train/test 不能重疊)
- ✓ 至少報三個指標 (Accuracy、Precision/Recall、F1)
- ✓ 有 Confusion Matrix 圖
- ✓ 至少對一個 baseline 做了對比
- ✓ 有失敗案例分析 (為什麼有些 case 會錯?)
- ✓ 有 limitations 章節 (自己承認模型在什麼條件下不夠好)
- ✓ 有 future work 章節 (你接下來怎麼改進)

2.5 為什麼在「嵌入式 CV」這特別重要

嵌入式 CV 不是「能跑就好」的領域。

一個 model 在伺服器上 95% accuracy + 100ms 延遲是 A 級，但搬到手機上變 70% + 800ms 就是 D 級。

因為嵌入式系統有資源約束：算力、記憶體、功耗、儲存空間都有上限。一個 paper 若不報這些 spec，工程上就不能用。

研究生畢業前必須學會：在資源約束下，用定量數據說明「我的方法可不可行」。

第 3 章 評估指標完整講解

3.1 分類任務指標全景圖

一張表先看完所有指標：

類別	指標	適用情境
精準型	Accuracy	類別均衡、總體性能
	Precision	「我說 Yes 中對的比例」
	Recall (Sensitivity)	「真的 Yes 中我抓到的比例」
	F1-score	Precision 與 Recall 的調和平均
整體型	Confusion Matrix	看清楚每一類錯在哪
	ROC / AUC	不同閾值下的整體表現
	macro / micro / weighted avg	多類別的綜合
嵌入式型	Inference latency (ms)	推論一張圖要多久
	Throughput (FPS)	每秒處理幾張圖
	Model size (MB)	模型佔多少磁碟空間
	Memory footprint (MB)	推論期間 RAM 峰值
	Power (mW)	功耗 (移動裝置才在意)

3.2 Accuracy 的陷阱 | 為什麼研究生不能只看 accuracy

Accuracy = 預測對的數量 / 總樣本數。看起來最直觀，但有個致命陷阱：

class 不均衡時 accuracy 會騙人。譬如 95% 都是「沒病」的醫療資料，模型只要永遠回答「沒病」就有 95% accuracy。

本講的 RPS 資料相對均衡（每類約 200 張），accuracy 還算可信。但研究級報告必須報多個指標。

3.3 Precision / Recall / F1-score

3.3.1 二類版定義

	預測 Positive	預測 Negative
真實 Positive	TP	FN
真實 Negative	FP	TN

Precision = $TP / (TP + FP)$ 我說 Yes 中對的比例

Recall = $TP / (TP + FN)$ 真的 Yes 中我抓到的比例

F1 = $2 * P * R / (P + R)$ 兩者調和平均

3.3.2 三者的物理意義

用「醫療診斷」比喻：

Precision 高 = 我說有病的人，幾乎都真的有病（很少誤診健康人）

Recall 高 = 真的有病的人，我幾乎都抓到了（很少漏診病人）

兩者本質衝突：要 Recall 高就放寬閾值 → Precision 會降；反之亦然

3.3.3 何時偏向 Precision 何時偏向 Recall

癌症篩檢 → 偏 Recall（寧可誤診健康人為陽性，也不能漏掉真的有癌的）

垃圾郵件過濾 → 偏 Precision（寧可漏掉一些垃圾，也不能把重要郵件丟進垃圾桶）

一般情境 → 用 F1 平衡兩者

3.3.4 多類別怎麼算（本講 RPS 是 3 類）

Precision / Recall 原本是二類定義。多類別有三種匯總方式：

macro average：每類各自算 P/R，最後平均 → 不考慮樣本數差異

micro average：把所有類別的 TP/FP/FN 加總後再算 → 受樣本數多的類別影響大

weighted average：每類算 P/R 後依樣本數加權 → 介於兩者之間

研究級報告通常三個都報。本講作業要求至少報 macro avg。

3.4 Confusion Matrix | 學生最該學會看的圖

3.4.1 3×3 Confusion Matrix 的讀法

	預測：石頭	預測：剪刀	預測：布
真實：石頭	85	5	10
真實：剪刀	3	78	19
真實：布	8	12	80

對角線 = 各類正確預測數量

非對角線 = 各種混淆模式

3.4.2 從 Confusion Matrix 看出什麼

上面這個矩陣告訴你：

石頭辨識最穩定（85/100 對）

剪刀和布最容易混淆（剪刀錯成布 19 次、布錯成剪刀 12 次）

→ 失敗案例分析時應該深入研究：為什麼剪刀和布在你資料集上混淆？

這種「具體錯在哪」的洞察，光看 accuracy 完全看不出來。

3.4.3 sklearn 算 Confusion Matrix

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt
```

```
cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(cm, display_labels=['Rock', 'Scissors', 'Paper'])
disp.plot(cmap='Blues')
plt.title('Confusion Matrix')
plt.show()
```

3.5 ROC / AUC 與多類別處理

ROC (Receiver Operating Characteristic) 是「不同分類閾值下，TPR vs FPR」的曲線。
AUC 是該曲線下方的面積，範圍 0-1，越大越好。

本講 RPS 為多類別，需用 One-vs-Rest 或 One-vs-One 拆成多個二類問題分別算 ROC。

對嵌入式即時應用，AUC 不是首要指標（不會調閾值，直接取 argmax）。研究級報告可以選報。

3.6 嵌入式專屬評估指標

3.6.1 Inference latency 與 Throughput

```
import time
```

```
# 量測單張推論時間
```

```
latencies = []
```

```
for _ in range(100): # 跑 100 次取平均
```

```
    t0 = time.perf_counter()
```

```
    _ = model.predict(x)
```

```
    latencies.append((time.perf_counter() - t0) * 1000) # 轉 ms
```

```
print(f'平均延遲: {np.mean(latencies):.2f} ms')
```

```
print(f'95% CI: {np.percentile(latencies, 2.5):.2f} - {np.percentile(latencies, 97.5):.2f} ms')
```

```
print(f'Throughput: {1000 / np.mean(latencies):.1f} FPS')
```

注意第一次推論通常較慢（warm-up），實測時記得棄掉。

3.6.2 Model size

import os

```
size_mb = os.path.getsize("rps_mobilenetv2.tflite") / (1024 * 1024)
print(f'模型大小: {size_mb:.2f} MB')
```

嵌入式系統 ROM 通常有限，模型大小直接影響「能不能塞進去」。

3.6.3 Memory footprint

量推論時 RAM 峰值，在 Python 上用 psutil 或 tracemalloc：

import psutil, os

```
process = psutil.Process(os.getpid())
print(f'推論前 RAM: {process.memory_info().rss / 1024**2:.1f} MB')
_ = model.predict(x)
print(f'推論後 RAM: {process.memory_info().rss / 1024**2:.1f} MB')
```

3.7 sklearn 與 TF 怎麼算這些指標（程式碼模板）

通用評估模板（sklearn）

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

y_pred = model.predict(X_test)

```
print(f'Accuracy: {accuracy_score(y_test, y_pred):.4f}')
```

```
print(classification_report(y_test, y_pred,
```

```
    target_names=['Rock', 'Scissors', 'Paper']))
```

```
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)
```

classification_report 一次給出 precision / recall / F1 / support，是研究級報告的基本配備。

第 4 章 資料集與前處理（Colab）

4.1 資料集介紹 | Sign Language Digits Dataset

來自 Kaggle 的開源資料集：

完整資料集：手語數字 0-9 共 10 類

每類約 200+ 張，總計約 2062 張

解析度 64×64，灰階

已被 Ardamavi 整理成 X.npy（影像）+ Y.npy（one-hot 標籤）

本講只保留三類：

0 = 拳頭 → 石頭 (Rock)
2 = 兩指伸出 → 剪刀 (Scissors)
5 = 五指張開 → 布 (Paper)

4.2 下載資料集 (Kaggle CLI)

在 Colab 第一個 cell 執行：

```
!pip install kaggle --quiet
```

```
# 上傳 kaggle.json ( 你的 API token )
from google.colab import files
files.upload() # 選 kaggle.json
```

```
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json
```

```
# 下載資料集
!kaggle datasets download -d ardamavi/sign-language-digits-dataset
!unzip -q sign-language-digits-dataset.zip
```

4.3 過濾並載入三類 (jpg 版)

```
import pathlib, cv2, numpy as np
```

```
KEEP_DIGITS = [0, 2, 5]
LABEL_MAP = {0: 0, 2: 1, 5: 2} # 重新編碼為 0,1,2
```

```
X, y = [], []
for d in KEEP_DIGITS:
    for p in pathlib.Path(f'Dataset/{d}').glob('*.jpg'):
        img = cv2.imread(str(p), cv2.IMREAD_GRAYSCALE)
        X.append(img)
        y.append(LABEL_MAP[d])
```

```
X = np.array(X, dtype=np.uint8) # (N, 64, 64)
y = np.array(y, dtype=np.int32) # (N,)
print(f'樣本數: {X.shape[0]}, 形狀: {X.shape[1:]}')
```

4.4 過濾並載入三類 (npy 版 , 較快)

```
X_all = np.load("X.npy") # (2062, 64, 64) 灰階
Y_all = np.load("Y.npy") # (2062, 10) one-hot
labels = np.argmax(Y_all, axis=1)
```

```
KEEP = [0, 2, 5]
LABEL_MAP = {0: 0, 2: 1, 5: 2}
keep_idx = np.isin(labels, KEEP)
X = X_all[keep_idx]
```



```
y = np.array([LABEL_MAP[v] for v in labels[keep_idx]])
print(f'樣本數: {X.shape[0]}, 三類分佈: {np.bincount(y)}')
```

4.5 資料分割 | train / val / test = 70 / 15 / 15

為什麼要分三份而不是兩份？

train 用來「訓練」模型參數

val 用來「調超參數」（學習率、epoch 數、模型大小等）

test 完全保留到最後一刻，只用來「報告最終性能」

如果只切 train / test，用 test 調超參數，等於偷看答案 — 報告的性能會被高估。

```
from sklearn.model_selection import train_test_split
```

```
# 先切出 test (15%)
```

```
X_trainval, X_test, y_trainval, y_test = train_test_split(
    X, y, test_size=0.15, stratify=y, random_state=42)
```

```
# 再從剩下的切 val (15% 原始 = 約 18% 剩餘)
```

```
X_train, X_val, y_train, y_val = train_test_split(
    X_trainval, y_trainval, test_size=0.18, stratify=y_trainval, random_state=42)
```

```
print(f'Train: {X_train.shape[0]}, Val: {X_val.shape[0]}, Test: {X_test.shape[0]}')
```

4.6 為什麼 stratify 很重要

stratify=y 確保每個 split 內三類的比例都跟原始資料一樣。

若不 stratify，可能 test set 大部分都是石頭，accuracy 看起來高但實際上不可信。

4.7 統一前處理

三種路線的前處理略有不同：

HoG + SVM：保留 64×64 灰階即可，HoG 自己處理特徵

Tiny CNN：64×64 灰階，normalize 到 [0, 1]，加 channel dim 變 (64, 64, 1)

MobileNetV2：64×64 灰階先 resize 到 96×96，灰階轉 RGB（複製 3 通道），normalize 到 [0, 1]

```
# 對 Tiny CNN
```

```
X_train_cnn = X_train.astype(np.float32) / 255.0
```

```
X_train_cnn = X_train_cnn[..., np.newaxis] # (N, 64, 64, 1)
```

```
# 對 MobileNetV2
```

```
import tensorflow as tf
```

```
X_train_mnv2 = tf.image.resize(X_train[..., np.newaxis], [96, 96])
```

```
X_train_mnv2 = tf.image.grayscale_to_rgb(X_train_mnv2)
```

```
X_train_mnv2 = X_train_mnv2.numpy().astype(np.float32) / 255.0
```

第 5 章 路線 B | HoG + Linear SVM

本路線：手工特徵 (HoG) + 學習式分類器 (Linear SVM)。

適合部署在：Android Chaquopy (sklearn 在 Chaquopy 上有 cp310 wheel，實測可運作)。

5.1 HoG 特徵簡介

HoG (Histogram of Oriented Gradients) 跟 W7 學的 SIFT/ORB descriptor 觀念類似：

把影像切成小 cell (譬如 8×8)

每 cell 內計算 9 個方向的梯度直方圖

用 2×2 cell 為一個 block 做歸一化

所有 block 串起來形成一個 N 維特徵向量

對 64×64 影像、cell 8×8 、9 方向、block 2×2 ，HoG 向量約 1764 維。

5.2 Linear SVM 直覺

SVM (Support Vector Machine) 的目標：找一個 hyperplane 把不同類別分開，使「邊界到最近樣本的距離 (margin)」最大化。

Linear SVM 假設可以用「直線」分開 (在高維是 hyperplane)。對 HoG 特徵效果不錯。

5.3 Colab 完整訓練流程

```
import numpy as np
from skimage.feature import hog
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import joblib
```

1. HoG 特徵提取

```
def img2hog(img):
    return hog(img,
               orientations=9,
               pixels_per_cell=(8, 8),
               cells_per_block=(2, 2),
               block_norm="L2-Hys")
```

對 train / val / test 都做

```
X_train_hog = np.array([img2hog(im) for im in X_train], dtype=np.float32)
X_val_hog = np.array([img2hog(im) for im in X_val], dtype=np.float32)
X_test_hog = np.array([img2hog(im) for im in X_test], dtype=np.float32)
print("HoG 向量維度 =", X_train_hog.shape[1])
```

2. 訓練 Linear SVM

```
clf = LinearSVC(max_iter=5000, C=1.0)
clf.fit(X_train_hog, y_train)
```

3. 評估

```
y_pred = clf.predict(X_test_hog)
```

```
print(f'Test Accuracy: {accuracy_score(y_test, y_pred):.4f}')
print(classification_report(y_test, y_pred,
    target_names=['Rock', 'Scissors', 'Paper']))

cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)
```

4. 儲存模型

```
joblib.dump(clf, "hog_svm_rps.pkl")
print("已儲存 hog_svm_rps.pkl")
```

5.4 預期結果

在 Sign Language Digits 0/2/5 三類上，HoG + Linear SVM 典型表現：

Test Accuracy: 92% - 96%

模型大小: 約 50 - 100 KB (極小)

Colab 上推論單張: 約 2 - 5 ms

5.5 注意：sklearn 版本相容性

Chaquopy 17 + Python 3.10 上的 scikit-learn 約是 1.3.x - 1.4.x 之間。Colab 上的 sklearn 通常更新，可能 1.5+。

版本差異有時會讓 joblib.load() 拋警告或失敗。建議做法：

在 Colab 上 pin 版本：!pip install "scikit-learn==1.4.0" --quiet

訓練前印出 sklearn.__version__ 記錄在報告中

若 Android 上載入失敗，先確認兩邊版本是否一致

5.6 Android Chaquopy 部署

5.6.1 app/build.gradle 設定

```
chaquopy {
    defaultConfig {
        version "3.10"
        pip {
            install "numpy"
            install "opencv-python"
            install "scikit-image"
            install "scikit-learn==1.4.0"
            install "joblib"
        }
    }
}
```

注意：scikit-learn 鎖死版本與 Colab 訓練端一致。

5.6.2 模型檔放置

把 Colab 訓練好的 hog_svm_rps.pkl 下載後，放到：

```
app/src/main/python/models/hog_svm_rps.pkl
```

5.6.3 Python 端 : hand_gesture_svm.py

```
# app/src/main/python/hand_gesture_svm.py
```

```
import os
```

```
import cv2
```

```
import numpy as np
```

```
from skimage.feature import hog
```

```
import joblib
```

```
# 用 __file__ 取絕對路徑 (這是 Chaquopy 上的必做步驟)
```

```
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
```

```
MODEL_PATH = os.path.join(BASE_DIR, "models", "hog_svm_rps.pkl")
```

```
CLF = joblib.load(MODEL_PATH)
```

```
LABELS = ["Rock", "Scissors", "Paper"]
```

```
# 膚色偵測 (沿用上一講 RPS 規則法的版本)
```

```
LOWER_SKIN = np.array([0, 30, 60], dtype=np.uint8)
```

```
UPPER_SKIN = np.array([20, 150, 255], dtype=np.uint8)
```

```
KERNEL = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
```

```
def detect_hand_mask(bgr):
```

```
    hsv = cv2.cvtColor(bgr, cv2.COLOR_BGR2HSV)
```

```
    mask = cv2.inRange(hsv, LOWER_SKIN, UPPER_SKIN)
```

```
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, KERNEL, iterations=2)
```

```
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, KERNEL, iterations=2)
```

```
    return mask
```

```
def largest_contour(mask):
```

```
    cnts, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,  
cv2.CHAIN_APPROX_SIMPLE)
```

```
    if not cnts:
```

```
        return None
```

```
    c = max(cnts, key=cv2.contourArea)
```

```
    return c if cv2.contourArea(c) > 3000 else None
```

```
def img2hog(img64):
```

```
    return hog(img64,
```

```
        orientations=9,
```

```
        pixels_per_cell=(8, 8),
```

```
        cells_per_block=(2, 2),
```

```
        block_norm="L2-Hys")
```

```
def process_nv21(nv21_bytes, w, h):
```

```

""""Chaquopy 入口 : Java 端 NV21 byte[] 進來 , 回傳 label 字串""""
yuv = np.frombuffer(bytes(nv21_bytes), dtype=np.uint8) \
    .reshape((h * 3 // 2, w))
bgr = cv2.cvtColor(yuv, cv2.COLOR_YUV2BGR_NV21)

# 1. 偵測手部 ROI
mask = detect_hand_mask(bgr)
cnt = largest_contour(mask)
if cnt is None:
    return "No hand detected"

# 2. 抓 bounding box , 轉灰階 , resize 到 64×64
x, y, ww, hh = cv2.boundingRect(cnt)
roi = bgr[y:y+hh, x:x+ww]
if roi.size == 0:
    return "No hand detected"
gray = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)
img64 = cv2.resize(gray, (64, 64))

# 3. HoG 特徵 + SVM 預測
feat = img2hog(img64)
pred = int(CLF.predict([feat])[0])
return LABELS[pred]

```

5.6.4 Java 端 : 在 ImageAnalysis 中呼叫

```

private void analyze(ImageProxy image) {
    frameCount++;
    if (frameCount % 5 != 0) { image.close(); return; }

    int w = image.getWidth();
    int h = image.getHeight();
    byte[] nv21 = imageProxyToNv21(image); // 沿用 W8 第 11 章
    image.close();

    PyObject module = py.getModule("hand_gesture_svm");
    PyObject result = module.callAttr("process_nv21", nv21, w, h);
    String label = result.toString();

    runOnUiThread(() -> labelView.setText("HoG+SVM: " + label));
}

```

5.7 常見問題

5.7.1 joblib serial mode 警告

Android 沒有完整的 multiprocessing 支援 , joblib 會印出 :

UserWarning: joblib will operate in serial mode

這是 warning 不是 error , 可以忽略。LinearSVC 推論不需要 parallel , 不影響結果。

5.7.2 sklearn 版本不一致警告

若 Colab 訓練時 sklearn 1.5 , Android 部署時 sklearn 1.4 , joblib.load() 可能拋 :

InconsistentVersionWarning: Trying to unpickle estimator LinearSVC from version 1.5...

多數情況下仍能正常運作。要根除 : 兩邊 pin 同一版本。

5.7.3 模型載入失敗

若拋 FileNotFoundError , 檢查 :

hog_svm_rps.pkl 是否真的在 app/src/main/python/models/ 路徑

Python 端是否用 __file__ 組路徑 (不能直接寫 "hog_svm_rps.pkl")

Android Gradle 是否重新 build (修改 python 目錄後常要 clean build)

5.8 預期 Android 上效能

Pi 4 / 中階手機 (Snapdragon 7 系列) : 每 frame 約 30 - 80 ms

FPS : 約 12 - 30 fps (含 HoG 計算時間)

APK 增量 : scikit-learn + scikit-image 約增加 25-40 MB

第 6 章 路線 C | Tiny CNN 自訓

本路線 : 用 Keras 自己設計小 CNN , 從像素端到端學特徵 + 分類。

適合部署在 : Colab Local Runtime + 本地 Webcam (tf-lite-runtime 在 Chaquopy 上沒 cp310 wheel) 。

6.1 為什麼從 Tiny CNN 起步

在跳到 MobileNetV2 之前 , 先用 Tiny CNN 讓學生看到「端到端從像素學特徵」的概念。

Tiny CNN 也是「sanity check」: 如果連 Tiny CNN 都跑不起來 , 後面 MobileNetV2 八成也有環境問題。

6.2 架構設計

Input (64, 64, 1) 灰階

↓

Conv2D(32, 3x3, ReLU) + MaxPool(2x2)

↓

Conv2D(64, 3x3, ReLU) + MaxPool(2x2)

↓

Flatten

↓

Dense(64, ReLU) + Dropout(0.3)

↓

Dense(3, softmax)

↓

Output (3 類機率)

約 50 萬參數 , model 大小 < 2 MB。

6.3 Colab 完整訓練流程

```
import tensorflow as tf
from tensorflow.keras import layers, models
import numpy as np

# 1. 前處理 ( 4 章 4.7 的 Tiny CNN 部分 )
X_train_cnn = X_train.astype(np.float32)[..., np.newaxis] / 255.0
X_val_cnn = X_val.astype(np.float32)[..., np.newaxis] / 255.0
X_test_cnn = X_test.astype(np.float32)[..., np.newaxis] / 255.0

y_train_oh = tf.keras.utils.to_categorical(y_train, 3)
y_val_oh = tf.keras.utils.to_categorical(y_val, 3)
y_test_oh = tf.keras.utils.to_categorical(y_test, 3)

# 2. 模型
model = models.Sequential([
    layers.Input(shape=(64, 64, 1)),
    layers.Conv2D(32, 3, activation='relu'),
    layers.MaxPooling2D(2),
    layers.Conv2D(64, 3, activation='relu'),
    layers.MaxPooling2D(2),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(3, activation='softmax'),
])
model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-3),
    loss='categorical_crossentropy',
    metrics=['accuracy'])
model.summary()

# 3. 訓練 ( 含 EarlyStopping )
early = tf.keras.callbacks.EarlyStopping(
    monitor='val_loss', patience=5, restore_best_weights=True)

history = model.fit(
    X_train_cnn, y_train_oh,
    validation_data=(X_val_cnn, y_val_oh),
    epochs=30, batch_size=32,
    callbacks=[early], verbose=2)

# 4. 評估
loss, acc = model.evaluate(X_test_cnn, y_test_oh, verbose=0)
print(f"Test Accuracy: {acc:.4f}")

y_pred = np.argmax(model.predict(X_test_cnn, verbose=0), axis=1)
from sklearn.metrics import classification_report, confusion_matrix
print(classification_report(y_test, y_pred,
```



```
target_names=['Rock', 'Scissors', 'Paper']))  
print(confusion_matrix(y_test, y_pred))
```

5. 儲存

```
model.save("rps_tiny_cnn.h5")
```

6.4 過擬合與 Dropout

Tiny CNN 有 50 萬參數，但資料只有約 600 張，模型容量遠大於資料 → 容易 overfit。

防 overfit 三招：

Dropout (本講已加 0.3) — 訓練時隨機讓 30% 神經元失活

Data Augmentation — 訓練時隨機翻轉、旋轉、縮放圖片，相當於擴增資料

EarlyStopping — 監測 val_loss，連續 5 epoch 不降就停 (本講已加)

6.5 訓練曲線判讀

用 matplotlib 畫 history 後，看四種典型情況：

train_loss 降、val_loss 也降 → 健康，繼續訓練

train_loss 降、val_loss 開始上升 → overfit，該停了

train_loss 不降 → 模型太小或學習率太大

train_loss 與 val_loss 都很高且平坦 → underfit，加大模型或調學習率

```
import matplotlib.pyplot as plt
```

```
plt.plot(history.history['loss'], label='train_loss')
```

```
plt.plot(history.history['val_loss'], label='val_loss')
```

```
plt.xlabel('Epoch'); plt.ylabel('Loss'); plt.legend(); plt.show()
```

6.6 轉換為 TFLite

```
converter = tf.lite.TFLiteConverter.from_keras_model(model)
```

```
tflite_model = converter.convert()
```

```
with open("rps_tiny_cnn.tflite", "wb") as f:
```

```
    f.write(tflite_model)
```

```
print("☐ 已輸出 rps_tiny_cnn.tflite")
```

6.7 預期結果

Test Accuracy: 88% - 94%

模型大小: 約 1.5 - 2 MB

Colab 上推論單張: 約 5 - 15 ms

6.8 Colab Local Runtime + Webcam 部署

回到你上一講教過的 Local Runtime 串流架構。先確認本地端已建好環境：

6.8.1 確認 Local Runtime 環境

Anaconda Prompt (本地端，不是 Colab)

```
conda activate colab_cv
```

```
pip install tensorflow opencv-python "numpy<2" jupyter_http_over_ws
```

```
jupyter server extension enable --py jupyter_http_over_ws
```

```
# 啟動 Jupyter ( 記得 CORS 設定 )
```

```
jupyter notebook --NotebookApp.allow_origin='https://colab.research.google.com' --  
NotebookApp.port_retries=0 --NotebookApp.token='' --  
NotebookApp.disable_check_xsrf=True --port=8888
```

Colab 右上角「連線」→ 選「連線至本地執行階段」→ 輸入 <http://localhost:8888/>

6.8.2 Local Runtime 上的即時推論程式

```
import cv2
```

```
import numpy as np
```

```
from tensorflow.keras.models import load_model
```

```
# 載入模型
```

```
model = load_model("rps_tiny_cnn.h5")
```

```
LABELS = ['Rock', 'Scissors', 'Paper']
```

```
# 開啟 webcam
```

```
cap = cv2.VideoCapture(0, cv2.CAP_DSHOW) # Windows: 強制 DirectShow
```

```
# 簡單膚色偵測 ( 同第 5 章 )
```

```
LOWER_SKIN = np.array([0, 30, 60], dtype=np.uint8)
```

```
UPPER_SKIN = np.array([20, 150, 255], dtype=np.uint8)
```

```
KERNEL = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
```

```
while True:
```

```
    ret, frame = cap.read()
```

```
    if not ret: break
```

```
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
```

```
    mask = cv2.inRange(hsv, LOWER_SKIN, UPPER_SKIN)
```

```
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, KERNEL, iterations=2)
```

```
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, KERNEL, iterations=2)
```

```
    cnts, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,  
cv2.CHAIN_APPROX_SIMPLE)
```

```
    if cnts:
```

```
        c = max(cnts, key=cv2.contourArea)
```

```
        if cv2.contourArea(c) > 3000:
```

```
            x, y, w, h = cv2.boundingRect(c)
```

```
            roi = frame[y:y+h, x:x+w]
```

```
            gray = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)
```

```
            img = cv2.resize(gray, (64, 64))[..., np.newaxis] / 255.0
```

```
            inp = np.expand_dims(img, 0).astype(np.float32)
```

```
            probs = model.predict(inp, verbose=0)[0]
```

```
            label = LABELS[int(np.argmax(probs))]
```

```
            cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
```

```
            cv2.putText(frame, f"{label} ({probs.max():.2f})",
```

```
                (x, y - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)
```

```
cv2.imshow("Tiny CNN RPS", frame)
if cv2.waitKey(1) & 0xFF == ord('q'): break
```

```
cap.release()
cv2.destroyAllWindows()
```

6.8.3 預期效能

本地端中階 PC (i5) : 約 30 - 60 FPS

完全本地化，無網路延遲

Webcam 解析度可以開到 720p、1080p 都流暢

第 7 章 路線 D | MobileNetV2 Transfer Learning (主菜)

本路線：用 ImageNet 預訓練的 MobileNetV2 當 backbone，只訓練分類層。

適合部署在：Colab Local Runtime + 本地 Webcam (同 Tiny CNN，避開 tf-lite-runtime cp310 問題)。

7.1 為什麼用 Transfer Learning

MobileNetV2 在 ImageNet 上訓練過 100 萬張影像，已經學到「邊緣、紋理、形狀」等低階特徵。

我們用它當 backbone，只訓練最後的分類層，等於「站在巨人肩膀上」：

資料少 (600 張) 也能訓練得好

訓練時間從幾小時降到幾分鐘

泛化能力比從零訓練好得多

7.2 MobileNetV2 架構簡介

MobileNetV2 的核心是「depthwise separable convolution」：把標準卷積拆成 depthwise + pointwise 兩步，運算量降到 1/8 到 1/9。

完整 MobileNetV2 約 3.5 百萬參數，model 8-15 MB (依量化程度)。比起 ResNet50 (98 MB) 小很多，適合嵌入式。

7.3 Colab 完整訓練流程

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping
import numpy as np
```

1. 前處理 (灰階 → 96×96 RGB)

```
def to_rgb96(X_gray):
    X = X_gray[..., np.newaxis].astype(np.float32)
    X = tf.image.resize(X, [96, 96])
```

```

X = tf.image.grayscale_to_rgb(X)
return X.numpy() / 255.0

X_train_d = to_rgb96(X_train)
X_val_d   = to_rgb96(X_val)
X_test_d  = to_rgb96(X_test)

y_train_oh = tf.keras.utils.to_categorical(y_train, 3)
y_val_oh   = tf.keras.utils.to_categorical(y_val,  3)
y_test_oh  = tf.keras.utils.to_categorical(y_test, 3)

# 2. 資料增強層
data_aug = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1),
])

# 3. MobileNetV2 基底 ( 凍結權重 )
base = tf.keras.applications.MobileNetV2(
    input_shape=(96, 96, 3),
    include_top=False,
    weights='imagenet')
base.trainable = False # 凍結 ( 先只訓練分類層 )

model = models.Sequential([
    layers.Input(shape=(96, 96, 3)),
    data_aug,
    base,
    layers.GlobalAveragePooling2D(),
    layers.Dropout(0.3),
    layers.Dense(3, activation='softmax'),
])
model.compile(
    optimizer=Adam(1e-3),
    loss='categorical_crossentropy',
    metrics=['accuracy'])

# 4. 第一階段訓練 ( 只訓練 head )
early = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
history1 = model.fit(
    X_train_d, y_train_oh,
    validation_data=(X_val_d, y_val_oh),
    epochs=15, batch_size=32,
    callbacks=[early], verbose=2)

# 5. ( 選 ) 第二階段 : 解凍 base , 用小學習率 fine-tune
# base.trainable = True
# model.compile(optimizer=Adam(1e-5), loss='categorical_crossentropy',

```

```

metrics=['accuracy'])
# history2 = model.fit(X_train_d, y_train_oh, validation_data=(X_val_d, y_val_oh),
#                       epochs=10, batch_size=32, callbacks=[early])

# 6. 評估
loss, acc = model.evaluate(X_test_d, y_test_oh, verbose=0)
print(f'Test Accuracy: {acc:.4f}')

y_pred = np.argmax(model.predict(X_test_d, verbose=0), axis=1)
from sklearn.metrics import classification_report, confusion_matrix
print(classification_report(y_test, y_pred,
    target_names=['Rock', 'Scissors', 'Paper']))
print(confusion_matrix(y_test, y_pred))

# 7. 儲存
model.save("rps_mobilenetv2.h5")

```

7.4 凍結預訓練權重的觀念 (`base.trainable = False`)

預訓練權重已經學到通用視覺特徵，我們不想讓「微小資料集」干擾它。

做法：`base.trainable = False`，訓練時只更新分類層 (`Dense(3, softmax)`) 的參數。

進階：第一階段訓練好後，可以解凍 `base`，用很小的學習率 (`1e-5`) 再 fine-tune 幾個 epoch。

7.5 灰階 → RGB 與 96×96 解析度

MobileNetV2 預設輸入是 RGB 三通道，但 Sign Language Digits 是灰階。

做法：`tf.image.grayscale_to_rgb` 把灰階複製成三通道。

96×96 是 MobileNetV2 較小的可用輸入 (其他選項：128, 160, 192, 224)。本講選 96 平衡效能與精度。

7.6 轉換為 TFLite (含量化選項)

7.6.1 浮點版 (最簡單)

```

converter = tf.lite.TFLiteConverter.from_keras_model(model)
tflite_model = converter.convert()
with open("rps_mobilenetv2.tflite", "wb") as f:
    f.write(tflite_model)
print(f'□ 浮點版大小: {len(tflite_model) / 1024**2:.2f} MB')

```

7.6.2 動態範圍量化 (推薦平衡點)

```

converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
tflite_quant = converter.convert()
with open("rps_mobilenetv2_quant.tflite", "wb") as f:
    f.write(tflite_quant)
print(f'□ 量化版大小: {len(tflite_quant) / 1024**2:.2f} MB')

```

7.6.3 int8 全量化 (最小但要 representative dataset)

```
def representative_data_gen():
    for i in range(min(100, len(X_train_d))):
        yield [X_train_d[i:i+1]]

converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.representative_dataset = representative_data_gen
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
converter.inference_input_type = tf.uint8
converter.inference_output_type = tf.uint8
tflite_int8 = converter.convert()
```

7.7 預期結果

Test Accuracy: 93% - 97%

浮點 model 大小: 約 9 MB

動態範圍量化: 約 3 MB (壓縮 3 倍)

int8 全量化: 約 2.5 MB (壓縮 3.6 倍 , 精度通常損失 < 2%)

7.8 Colab Local Runtime + Webcam 部署 (Keras 與 TFLite 兩種寫法)

7.8.1 用 Keras model 推論 (簡單但慢)

```
from tensorflow.keras.models import load_model
model = load_model("rps_mobilenetv2.h5")

def predict_keras(roi_bgr):
    roi_rgb = cv2.cvtColor(roi_bgr, cv2.COLOR_BGR2RGB)
    roi_rgb = cv2.resize(roi_rgb, (96, 96))
    inp = np.expand_dims(roi_rgb / 255.0, 0).astype(np.float32)
    probs = model.predict(inp, verbose=0)[0]
    return LABELS[int(np.argmax(probs))], float(probs.max())
```

7.8.2 用 TFLite 推論 (快且穩定)

```
interpreter = tf.lite.Interpreter(model_path="rps_mobilenetv2.tflite")
interpreter.allocate_tensors()
IN_IDX = interpreter.get_input_details()[0]["index"]
OUT_IDX = interpreter.get_output_details()[0]["index"]

def predict_tflite(roi_bgr):
    roi_rgb = cv2.cvtColor(roi_bgr, cv2.COLOR_BGR2RGB)
    roi_rgb = cv2.resize(roi_rgb, (96, 96))
    inp = np.expand_dims(roi_rgb / 255.0, 0).astype(np.float32)
    interpreter.set_tensor(IN_IDX, inp)
    interpreter.invoke()
    probs = interpreter.get_tensor(OUT_IDX)[0]
    return LABELS[int(np.argmax(probs))], float(probs.max())
```

7.8.3 完整即時 webcam 程式 (用 TFLite 版)

```
cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)
LOWER_SKIN = np.array([0, 30, 60], dtype=np.uint8)
UPPER_SKIN = np.array([20, 150, 255], dtype=np.uint8)
KERNEL = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
LABELS = ['Rock', 'Scissors', 'Paper']

while True:
    ret, frame = cap.read()
    if not ret: break
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
    mask = cv2.inRange(hsv, LOWER_SKIN, UPPER_SKIN)
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, KERNEL, iterations=2)
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, KERNEL, iterations=2)
    cnts, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
    if cnts:
        c = max(cnts, key=cv2.contourArea)
        if cv2.contourArea(c) > 3000:
            x, y, w, h = cv2.boundingRect(c)
            roi = frame[y:y+h, x:x+w]
            if roi.size > 0:
                label, conf = predict_tflite(roi)
                cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
                cv2.putText(frame, f"{label} ({conf:.2f})",
                    (x, y-10), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0,255,0), 2)
            cv2.imshow("MobileNetV2 RPS", frame)
            if cv2.waitKey(1) & 0xFF == ord('q'): break
    cap.release()
cv2.destroyAllWindows()
```

7.9 為什麼 Local Runtime + TFLite 而不是 Android

如本講第 1 章所說：Chaquopy 官方 pypi-13.1 上的 tflite-runtime 只有 cp38 wheel (Python 3.8)，與 Chaquopy 17 的 Python 3.10 不相容。

硬編 wheel 對教學情境太重。Local Runtime 完全本地化 (影像不過網路)、效能極佳、與你既有教學流程一致，是研究 prototype 的最佳平台。

真要部署到 Android，下一講會走 Java TFLite + AAR 路徑 (YOLO + Tracking 章節)。

第 8 章 兩種部署平台的對比與工程意義

8.1 為什麼 ML 能 Chaquopy，DL 不能？

這是本講最重要的工程洞察：

部署成功與否，不是看模型強不強，而是看執行環境有沒有對應的 wheel / API。

8.2 wheel 可用性對照表

套件	Chaquopy 17 + Py3.10	備註
numpy	✓ 有 cp310 wheel	基礎
opencv-python	✓ 有 cp310 wheel	基礎
scikit-image	✓ 有 cp310 wheel	HoG 用
scikit-learn	✓ 有 cp310 wheel	SVM 用
joblib	✓ pure-Python	從 PyPI 裝
tf-lite-runtime	✗ 只有 cp38 wheel	無法用！
tensorflow	✗ 太大 + 已停更	無法用
torch / torchvision	✗ 已停更	無法用

8.3 工程上的兩個選擇

8.3.1 選擇 A：硬上 — 自己編 wheel

用 chaquo/build-wheel 工具自編 tf-lite-runtime cp310

需要 Linux x86-64 + Android NDK + Bazel

每個學生 build.gradle 改 --find-links 指向你的 wheel server

每次 TF 升版要重編一次

教學情境下太重，學生踩坑風險高。

8.3.2 選擇 B：分而治之 — 不同模型不同部署平台

本講採用的策略：

ML (HoG+SVM) 有 wheel → 部署 Android Chaquopy

DL (TFLite) 沒 wheel → 部署 Colab Local Runtime

真要 Android 跑 DL → 改用 Java TFLite API + AAR (下一講 YOLO + Tracking)

8.4 Local Runtime 為什麼是研究 prototype 的好平台

沿用你前面教過的「網路 vs 本地」對比：

影像流不過網路 → 不消耗 bandwidth，不受 Colab 連線品質影響

完全本地化 → 延遲極低，可達 30+ FPS

直接呼叫本地 OpenCV → 任何相機都能用 (包含高階 SID)

用 Colab 介面操控 → 程式碼仍在雲端管理，方便版本控制與分享

研究 prototype 階段，Local Runtime 是最有效率的平台。要追求商業級部署再轉 Android Java + AAR。

8.5 對嵌入式 CV 工程師的啟示

部署不是「模型好就好」，是「工具鏈完整才好」

選工具不是看「最潮的」，而是看「真的能跑的」

好的工程師會在第一天就盤點「我能用什麼套件」，再回頭挑模型

不要把所有運算都塞進 Python — 必要時讓 Java 接手是業界 hybrid 架構標準

8.6 下一講預告

下一講「YOLO + Tracking」會處理：

用 Android 官方 Java TFLite API (org.tensorflow:tensorflow-lite) 載入 YOLO

完全跳開 Python wheel 限制

Python (Chaquopy) 只負責前後處理, Java 負責推論

整合 tracking-by-detection 範式 (DeepSORT 思路)

第 9 章 三方法 × 傳統 CV 在系統上實測對比 (研究級對比)

9.1 為什麼要做這個對比？

這是把本講第 2 章「研究 vs 功能實作」的方法論，在本講內實踐一次。

學生會學到：「研究級報告長什麼樣」、「定量對比怎麼寫」、「結論怎麼下」。

9.2 實驗 spec (必須在報告中列明)

硬體：

PC : i5-10300H, 16GB RAM, GTX 1650 (Colab 訓練則用 T4 GPU)

Webcam : USB 2.0, 720p

Android 手機 (HoG+SVM) : Samsung A53, Snapdragon 778G, 8GB RAM

軟體：

Python 3.10, TensorFlow 2.15, scikit-learn 1.4.0

OpenCV 4.8, scikit-image 0.21

Chaquopy 17.0 (HoG+SVM 部署)

運行條件：

室內螢光燈, 約 500 lux

白色背景

手距鏡頭 30 - 50 cm

Webcam 解析度 640×480

評估：

Test set: 92 張獨立樣本 (從原資料集切 15%)

每方法跑 100 次推論取平均延遲

9.3 Confusion Matrix 四方法並列

四個矩陣 (規則法 / HoG+SVM / Tiny CNN / MobileNetV2) 並列展示，學生用 matplotlib subplot 排版：

```
import matplotlib.pyplot as plt
```

```
from sklearn.metrics import ConfusionMatrixDisplay, confusion_matrix
```

```
fig, axes = plt.subplots(1, 4, figsize=(20, 5))  
methods = {
```

```

'Rule-based': cm_rule,
'HoG + SVM': cm_svm,
'Tiny CNN': cm_cnn,
'MobileNetV2': cm_mnv2,
}
for ax, (name, cm) in zip(axes, methods.items()):
    disp = ConfusionMatrixDisplay(cm, display_labels=['Rock', 'Scissors', 'Paper'])
    disp.plot(ax=ax, cmap='Blues', colorbar=False)
    ax.set_title(name)
plt.tight_layout()
plt.savefig('confusion_matrix_compare.png', dpi=150)
plt.show()

```

9.4 預期對比表（學生作業要填真實數字）

指標 規則法 HoG+SVM Tiny CNN MobileNetV2

Accuracy	~70%	~94%	~92%	~96%
Precision (macro)	~68%	~94%	~92%	~96%
Recall (macro)	~70%	~94%	~92%	~96%
F1-score (macro)	~68%	~94%	~92%	~96%
推論延遲 (ms)	~25	~10	~12	~25
FPS	~40	~80	~70	~35
模型大小 (MB)	0	0.1	1.8	9.0
記憶體峰值 (MB)	50	80	120	180
部署平台	Android	Android	Local	Local
	Chaquopy	Runtime	Runtime	

上面數字為典型推估。作業要求學生實測填入「具有 95% 信心區間的真實數字」。

9.5 失敗案例分析

研究級報告必須含這一節 — 不是只報好結果。建議列出：

規則法：手指角度不標準時，剪刀和布常混淆

HoG+SVM：背景複雜時，HoG 抽出的特徵被污染

Tiny CNN：訓練資料背景單純，學到「白色背景 + 手」→ 換場景失敗

MobileNetV2：因為遷移學習，泛化最好，但對極端光線仍會錯

在報告中為每個方法挑 2-3 張具體失敗的圖，做視覺化分析。

9.6 從這個對比能寫出什麼樣的研究結論

看著上面的數字，研究生應該能寫出這樣的結論段：

「在 Sign Language Digits 0/2/5 三類任務上，MobileNetV2 Transfer Learning 在 accuracy 上勝過 HoG+SVM 約 2 個百分點，但推論延遲多 2.5 倍、模型大小多 90 倍。對於資源充裕的研究端（PC + Webcam）建議使用 MobileNetV2；對於資源受限的 Android 嵌入式端，HoG+SVM 是更實用的選擇（部署簡單、模型小、延遲低）。規則法雖然 accuracy 最低，但完全不需要訓練資料，適合 cold-start 階段或無資料情境。」

這段結論的特徵：有定量數字、有條件限定、有實務建議、不過度宣稱。

9.7 對下一講 Android 部署的展望

本講做完，學生會體會到：「最強的模型不一定能部署到嵌入式」。下一講 YOLO + Tracking 會：
用業界主流的「Java TFLite + AAR」路徑徹底解決 DL 模型上 Android 的問題
引入 tracking-by-detection 範式 (DeepSORT、ByteTrack)
在 RPS 之外，做出可以追蹤雙手、連續手勢的應用

第 10 章 本講作業與常見錯誤排除

10.1 基本要求 (過關)

完整跑完三條路線的訓練 (Colab notebook 必附)
每條路線都出 Test Accuracy、Confusion Matrix、classification_report
HoG+SVM 部署到 Android Chaquopy，成功運作 (含示範影片)
Tiny CNN 與 MobileNetV2 都在 Local Runtime + Webcam 上即時 demo 成功

10.2 加分項

- (+1) 完整研究級評估報告 (含實驗 spec、定量數字、失敗案例、limitations)
- (+2) 量化版 MobileNetV2 與浮點版的對比實測 (accuracy 與 size)
- (+1) 用 Teachable Machine 收自己的資料訓練 + 比較公開資料集
- (+1) 整合本講三方法與上一講規則法做多數決投票，看 accuracy 是否提升

10.3 繳交內容

Colab notebook (三條路線的訓練 .ipynb)
Android Studio 專案 zip (HoG+SVM 版本)
研究級評估報告 (pdf)
三段示範影片各 30 秒 (HoG+SVM Android、Tiny CNN Local Runtime、MobileNetV2 Local Runtime)

10.4 Colab 訓練常見錯誤

10.4.1 Kaggle CLI 失敗

成因：

kaggle.json 沒上傳到正確位置 (~/.kaggle/)
權限沒設 600
Kaggle API token 過期

10.4.2 NumPy 2.x 不相容

TensorFlow 2.15 還沒完全支援 NumPy 2.x。若拋 ImportError 或型別錯誤，pin 版本：
`!pip install "numpy<2" --quiet`

10.4.3 GPU 不可用 / OOM

在 Colab 設定 → Runtime → 加速器 → 選 T4 GPU。MobileNetV2 訓練 batch_size 設 32 通常不會 OOM。

10.5 Android Chaquopy 部署常見錯誤

10.5.1 joblib serial mode warning

正常，可忽略。

10.5.2 sklearn 版本不一致 warning

Colab pin 與 Chaquopy pin 都設 `scikit-learn==1.4.0`。

10.5.3 模型載入失敗

檢查 `hog_svm_rps.pkl` 是否在 `app/src/main/python/models/`

檢查 Python 端是否用 `__file__` 組路徑

Gradle 修改 python 目錄後務必 `clean build`

10.5.4 APK 太大

`scikit-learn` + `scikit-image` 約增加 30-40 MB。建議：

release 版只保留 `arm64-v8a` ABI (`abiFilters`)

開啟 `chaquopy { pyc { src true } }`

10.6 Colab Local Runtime 常見錯誤

10.6.1 連線失敗

Anaconda Prompt 是否仍在執行 `jupyter notebook` (不能關)

CORS 設定是否完整 (`--NotebookApp.allow_origin`)

port 8888 是否被佔 (換 `--port=8889` 試試)

10.6.2 cv2.VideoCapture 開不到攝影機

關閉所有用相機的 App (Zoom、OBS、Teams)

關閉瀏覽器的「相機授權」(彈出框中關閉)

Windows 上用 `cv2.CAP_DSHOW` 而不是 `cv2.CAP_ANY`

用本講附錄的「攝影機掃描程式」找正確 Index

10.6.3 NumPy 2.x 與 pandas 不相容

本地 Anaconda 環境一律 `pip install "numpy<2"`。

附錄 Colab Local Runtime + Webcam 設定 checklist

A.1 本地端環境 (一次性設定)

1. 建立獨立虛擬環境

```
conda create -n colab_cv python=3.10
```

```
conda activate colab_cv
```

2. 安裝核心套件

```
pip install opencv-python "numpy<2" tensorflow scikit-learn scikit-image joblib
```

```
pip install jupyter_http_over_ws
```

```
jupyter server extension enable --py jupyter_http_over_ws
```

A.2 每次啟動 Local Runtime

```
# Anaconda Prompt 中 ( 保持開啟 )  
conda activate colab_cv
```

```
jupyter notebook --NotebookApp.allow_origin='https://colab.research.google.com' \  
    --NotebookApp.port_retries=0 \  
    --NotebookApp.token="" \  
    --NotebookApp.disable_check_xsrf=True \  
    --port=8888
```

A.3 Colab 連線

右上角「連線」→ 下拉箭頭 → 「連線至本地執行階段」

輸入 <http://localhost:8888/>

右上角圖示變綠色勾勾 = 連線成功

A.4 攝影機掃描程式 (找正確 Index)

```
import cv2
```

```
def scan_cameras():  
    print("正在掃描本地可用攝影機...\n")  
    found = []  
    for i in range(6):  
        # Windows 用 DirectShow , macOS/Linux 拿掉第二參數  
        cap = cv2.VideoCapture(i, cv2.CAP_DSHOW)  
        if cap.isOpened():  
            w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))  
            h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))  
            found.append((i, w, h))  
            cap.release()  
  
    if not found:  
        print("❌ 找不到攝影機。請確認硬體連線、是否被其他程式佔用。")  
        return  
  
    print("--- 偵測到的攝影機 ---")  
    for idx, w, h in found:  
        tag = "[高解析]" if w >= 2880 else "[標準]"  
        print(f"Index [{idx}]: {w} x {h} -> {tag}")
```

```
scan_cameras()
```

A.5 常見地雷

- ⚠ 執行前必須關閉瀏覽器的網址列相機授權，否則資源衝突
- ⚠ 不要同時開 Zoom / OBS / Teams 等用相機的程式
- ⚠ NumPy 2.x 與 pandas 可能不相容，pin "numpy<2"
- ⚠ Anaconda Prompt 視窗關了 Local Runtime 就斷線